

openCollector: An Open-Source Edge-to-Cloud Platform for Environmental Sensor Data

Nathaniel King, Michael Chen, Christianna Sequeira, Phu Trung Le, and Stephen Makonin

Simon Fraser University, British Columbia, Canada

Corresponding author: Stephen Makonin (smakonin@sfu.ca)

Abstract—Environmental monitoring often requires integrating heterogeneous sensors, resource-constrained edge hardware, persistent time-series storage, and an accessible user interface. Existing research platforms can be difficult to reproduce or adapt when hardware drivers, data models, and deployment procedures are tightly coupled. This paper presents openCollector, an open-source edge-to-cloud platform developed around a Raspberry Pi Pico W collector, MicroPython-compatible sensor drivers, QuestDB storage, and a Python web service. The design separates sensing, transport, persistence, and presentation while retaining shared metadata across layers. The resulting prototype provides a practical foundation for research deployments that need inspectable hardware and software rather than a closed monitoring stack.

Index Terms—environmental monitoring, Internet of Things, edge computing, open-source hardware, sensor networks, time-series database

I. INTRODUCTION

Low-cost environmental sensor networks can increase the spatial and temporal resolution available to researchers, but the apparent simplicity of attaching a sensor to a networked microcontroller hides substantial systems work. A usable deployment must configure heterogeneous interfaces, associate measurements with stable metadata, tolerate communication failures, store time-indexed values, and make system status intelligible to users. A review of environmental sensor networks identifies openness, maintainability, calibration, data quality, cost and connectivity as persistent challenges [1]–[3].

The Internet of Things (IoT) is an architecture that connects physical sensing, network transport, distributed computation, and applications [4], [5]. Edge computing moves selected functions closer to data sources, reducing dependence on centralized cloud resources and enabling local control [6], [7]. However, many environmental projects remain application-specific assemblies of firmware, scripts, and cloud services. Reproducing or modifying them can require rebuilding the complete stack.

Existing low-cost environmental platforms commonly optimize only part of the measurement path. Single-purpose data loggers often couple sensor firmware to a particular storage representation, while general IoT platforms can conceal transport, persistence, and failure behaviour behind managed services. For research deployments, this makes it difficult to trace how a measurement is identified, transported, stored, and interpreted, or to determine what is lost when a component fails. openCollector addresses this reproducibility gap through

an inspectable, replaceable edge-to-cloud architecture that preserves explicit sensor and record identities across its layers.

This paper presents *openCollector*, an open-source platform intended to make that stack explicit and replaceable. Its main contributions are: (1) a configuration-driven collector and common sensor abstraction spanning Python and MicroPython; (2) a compact record and metadata model for time-series ingestion; (3) a separable storage, API, and browser layer; and (4) an open hardware and deployment workflow suitable for extension. The prototype originated as a 2024–2025 undergraduate capstone project from the SFU School for Sustainable Energy Engineering (SEE).

II. BACKGROUND

Early habitat-monitoring deployments established the value of autonomous, networked sensing while exposing constraints in energy, communication, and field maintenance [8]. Contemporary low-cost networks add Wi-Fi microcontrollers, web APIs, and commodity databases, but heterogeneous observations still require consistent identity and semantics. The OGC SensorThings API defines a Web-oriented observation model [9], while the W3C Web of Things architecture describes interoperable abstractions for connected devices [10]. openCollector does not yet implement either standard as a choice to meet a lightweight, cost-effective, architecturally simple design.

The edge node uses the Raspberry Pi Pico W, an RP2040-based microcontroller with 2.4-GHz wireless networking and exposed digital interfaces [11]. MicroPython provides a compact Python implementation for microcontrollers [12]. These choices allow much of the collector logic and its drivers to retain a common programming model on both microcontroller and desktop test environments. The platform uses QuestDB for time-series persistence and its line protocol for ingestion [13]; a Litestar application exposes queries and server-rendered views [14]; and Bokeh supplies interactive plots [15].

This combination occupies a middle ground between a single-purpose data logger and a general IoT platform. A logger can be simple to deploy but usually couples its storage format to one device. General platforms offer device management and analytics but can conceal the measurement path behind managed services. openCollector instead keeps the acquisition code small while making the database schema and service queries inspectable. It deliberately stores both observations and operating errors: a missing interval can

TABLE I
ARCHITECTURAL ELEMENTS AND DESIGN RATIONALE

Element	Selection	Rationale
Edge node	Pico W / MicroPython	Low-cost Wi-Fi device with Python-compatible firmware
Sensor layer	Driver interface	Isolates bus protocols and measurement schemas
Transport	QuestDB line protocol	Compact batched ingestion of timestamped values
Persistence	QuestDB	Time partitions, symbolic identifiers, SQL queries
Service	Litestar / Bokeh	Typed Python API and interactive browser plots
Deployment	Docker Compose	Repeatable database and application configuration

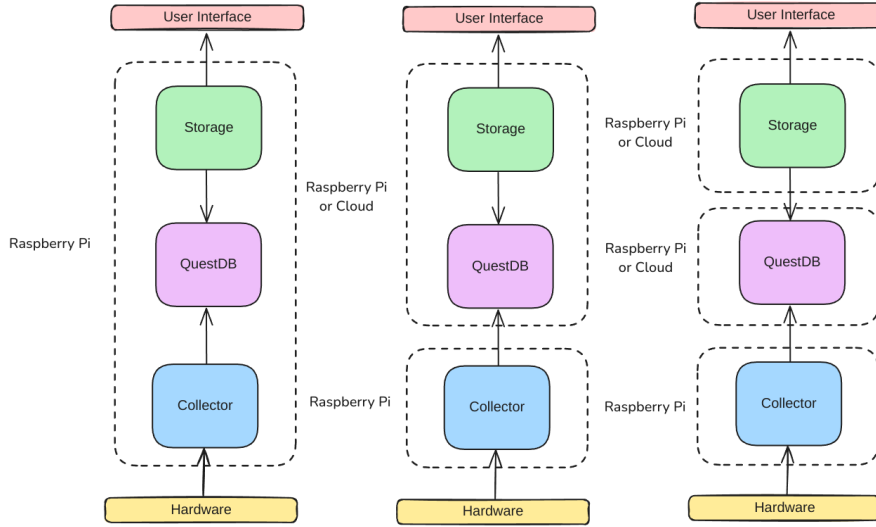


Fig. 1. Supported deployment options. The collector, QuestDB, and storage service may share a host or be placed on separate edge/server devices.

TABLE II
SENSORS IMPLEMENTED IN THE PROTOTYPE

Sensor	Interface	Measurements	Unit
DHT20	I ² C	Temperature, humidity	°C, %
TSL2561	I ² C	Illuminance	lx
PMS5003	UART	PM1.0, PM2.5, PM10	μg/m ³
MH-Z19B	UART	CO ₂ concentration	ppm

therefore be distinguished from a recorded sensor or upload failure when diagnosing a field deployment.

III. DESIGN METHODOLOGY

The design followed a requirements-led capstone process: identify stakeholders and measurements, compare candidate hardware and software, select a modular architecture, implement a vertical prototype, and document the result for later teams. Five requirements guided trade-offs: affordability, support for multiple sensor buses, deployability on constrained hardware, inspectable data flow, and adaptability beyond the initial greenhouse use case. Open-source licensing and reproducible fabrication were treated as design constraints rather than release tasks. This approach is consistent with open-source hardware principles, which require preferred design sources to be available for study and modification [16].

Figure 1 shows three supported placement patterns. The collector, database, and service can share a Raspberry Pi-class host during development; the collector can be separated from a server or cloud host; or all three elements can be independently placed. Each collector currently has one database target, whereas multiple collectors can feed one database. Separating the layers makes sensor timing independent of visualization and allows storage or presentation technologies to change without rewriting sensor drivers.

Table I summarizes the selected responsibilities. The collector uploads metadata at startup and then polls all configured sensors at a millisecond interval. A measurement is represented by collector, sensor, and record identifiers, a timestamp, and a numeric value. Metadata is stored separately so repeated records remain compact. Polling and upload failures are represented explicitly rather than being silently discarded.

Alternative choices were evaluated at each boundary. A Linux single-board computer simplifies driver development, but a microcontroller better represents the memory and deployment constraints expected at the sensing edge. A topic-oriented broker such as MQTT is attractive for intermittently connected networks, whereas direct line-protocol ingestion reduced the number of services needed for the first vertical prototype. Similarly, a fixed table per sensor would provide strongly typed columns but make new drivers a database migration. The chosen narrow record table stores numeric values by record

identifier and moves human-readable names into metadata. This favours extensibility and compact ingestion; the trade-off is that units and record semantics must be resolved through sensor metadata or the shared software definitions.

A further design requirement was compactness and portability. A custom sensor-interface PCB was developed to consolidate the Raspberry Pi Pico W, sensor connections, and supporting electronics, reducing reliance on loose breakout boards and jumper wiring while improving assembly repeatability. The PCB and sensor modules were arranged to fit within a purpose-designed, 3D-printable enclosure. The enclosure protects the electronics while providing access to power and programming connections and appropriate openings for airflow and optical exposure. Together, the PCB and enclosure create a self-contained sensing unit that can be transported, installed, serviced, and reproduced using commonly available fabrication equipment. These design files are released as open source with the project.

IV. SYSTEM IMPLEMENTATION

A. Collector and Sensor Interfaces

The collector comprises configuration, sensor, report, and upload managers. A JSON configuration supplies a collector UUID, display name, polling interval, Wi-Fi credentials, database endpoint, and an array of sensor definitions. The program connects to Wi-Fi, validates configuration, constructs drivers, uploads metadata, and enters a repeated poll-report-upload cycle. The current collector provides best-effort, at-most-once delivery. Each report is retained only for its active polling cycle and is submitted once. If an HTTP upload raises an exception, the exception is logged locally and the polling loop proceeds to the next cycle. The failed report is neither buffered nor retried; consequently, measurements produced during the failure are permanently absent from QuestDB. Sensor polling errors can be stored when communication is available, whereas communication failures themselves are not persisted remotely. Persistent offline buffering and bounded retry are therefore identified as future work.

Validation occurs before acquisition begins. Required keys are checked, sensor codes are mapped to registered driver classes, and each driver interprets only its own GPIO and optional attributes. This confines device-specific knowledge to the driver boundary. Implementations are paired where necessary: MicroPython drivers use the microcontroller's machine interfaces, while desktop Python variants and a mock driver permit development without attached hardware. The manager returns records and errors as separate collections and the report object binds them to the collector UUID. The upload layer batches the resulting lines, reducing per-value protocol overhead while preserving sensor and record identifiers.

Supported drivers are listed in Table II. The DHT20 and TSL2561 share an I²C bus, whereas the PMS5003 and MH-Z19B use UART. Their manufacturer documentation defines the electrical interfaces and payload interpretation [17]–[20]. Driver classes return named values with units or structured errors, enabling the same upload path to handle different

measurement cardinalities. Note that the system architecture easily allows new sensors to be added to data collection.

B. Persistence, API, and Dashboard

QuestDB tables store records, sensor errors, collector metadata, and sensor metadata. Time-series tables are partitioned by day; metadata tables are partitioned by week. Symbol columns compact repeated UUIDs and sensor codes. The storage service queries current totals, latest values, timelines, and error states. Browser routes render dashboard, collector-detail, and sensor-detail views, while REST routes expose the same repository layer. Authentication can be enabled for browser and API access, and production deployment places the service behind HTTPS. Container configuration follows the Compose specification [21].

The repository layer centralizes SQL rather than embedding it in page handlers. Consequently, browser fragments and API commands share the same definitions of “latest,” status, and timeline selection. Collector and sensor detail pages derive their identity from metadata and join it with the most recent observation or error. The dashboard can therefore show an online summary without copying descriptive fields into every measurement. For network deployment, the database ingestion port is restricted to collector networks, while users reach the web service through an HTTPS reverse proxy. Basic authentication protects the prototype interface but is not presented as a complete multi-tenant identity system.

Figure 2 shows the prototype dashboard populated with test data. It combines network-level totals, per-sensor status, latest values, and a selectable time-series plot. This view demonstrates end-to-end schema flow; it is not an accuracy or field-performance evaluation.

C. Source Code and Reproducibility

Source code, hardware design files, deployment configurations, documentation, and automated tests are available at <https://github.com/compsust/openCollector>.

V. CONCLUSIONS

openCollector demonstrates a complete, inspectable path from heterogeneous environmental sensors to time-series storage, APIs, and visualization. Its main value is architectural: hardware drivers, acquisition, storage, and presentation are independently replaceable but share explicit identifiers and metadata. The current implementation supports four sensor models, desktop-testable driver paths, repeatable service deployment, and public documentation. It should be understood as a research prototype rather than a calibrated instrument or a completed scalability study.

A. Future Work

The immediate priority is a controlled field evaluation reporting sample loss, end-to-end latency, database throughput, WiFi recovery, and multi-day stability across increasing collector counts. Energy experiments will characterize average and peak power, battery runtime, recharge requirements,

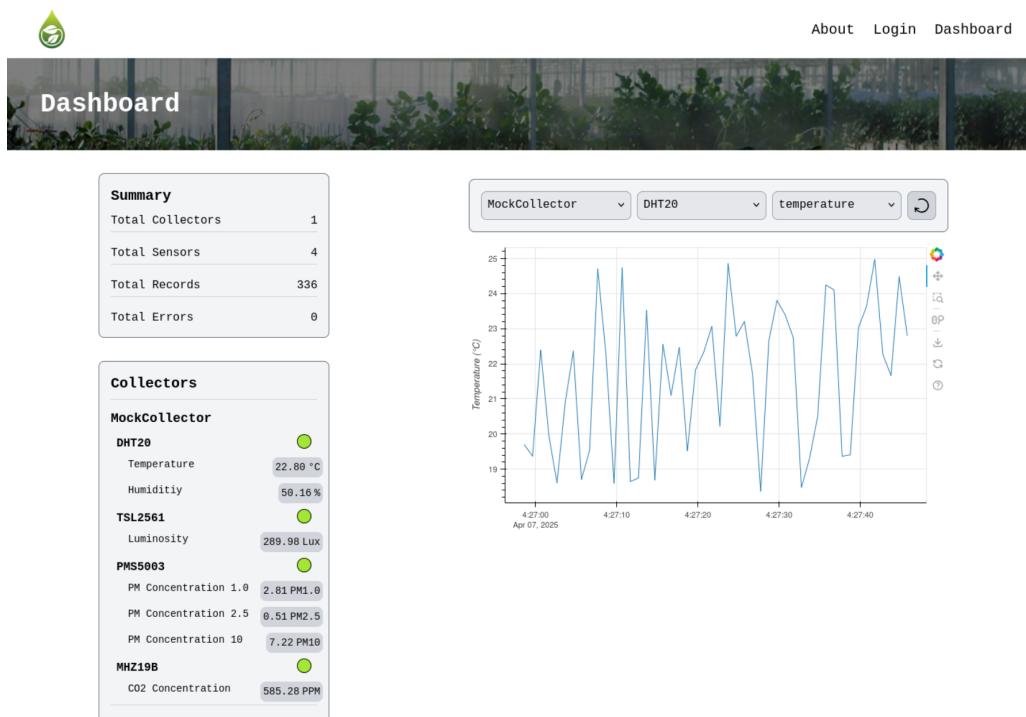


Fig. 2. Prototype dashboard populated with test data, showing network summaries, sensor status, latest values, and an interactive timeline.

and service continuity under battery-only and electric-vehicle-powered operation. Software work will add offline buffering, secure per-node credentials, over-the-air updates, and completed Pico W hardware-in-the-loop tests.

ACKNOWLEDGMENT

The authors thank Dr. Mina Xu and Dr. Manpreet Kaur, who co-instructed the SFU SEE 410W Fall 2024 capstone course, for their support of the project. This work was supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) through Discovery Grant RGPIN-2018-06192.

REFERENCES

- [1] J. K. Hart and K. Martinez, "Environmental sensor networks: A revolution in the earth system science?" *Earth-Science Reviews*, vol. 78, no. 3–4, pp. 177–191, 2006.
- [2] P. Corke, T. Wark, R. Jurdak, W. Hu, P. Valencia, and D. Moore, "Environmental wireless sensor networks," *Proceedings of the IEEE*, vol. 98, no. 11, pp. 1903–1917, 2010.
- [3] F. Mao, K. Khamis, S. Krause, J. Clark, and D. M. Hannah, "Low-cost environmental sensor networks: Recent advances and future directions," *Frontiers in Earth Science*, vol. 7, p. 221, 2019.
- [4] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [5] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of things (IoT): A vision, architectural elements, and future directions," *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, 2013.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [7] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [8] A. Mainwaring, D. Culler, J. Polastre, R. Szewczyk, and J. Anderson, "Wireless sensor networks for habitat monitoring," in *Proc. 1st ACM Int. Workshop Wireless Sensor Networks and Applications*, 2002, pp. 88–97.
- [9] Open Geospatial Consortium, "OGC SensorThings API Part 1: Sensing Version 1.1," Open Geospatial Consortium, Tech. Rep. 18-088, 2021. [Online]. Available: <https://docs.ogc.org/is/18-088/18-088.html>
- [10] World Wide Web Consortium, "Web of things (WoT) architecture 1.1," W3C, Tech. Rep., 2023. [Online]. Available: <https://www.w3.org/TR/wot-architecture11/>
- [11] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*, 2026, accessed: 2026-07-27. [Online]. Available: <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>
- [12] MicroPython Project, *MicroPython Documentation*, 2026, accessed: 2026-07-27. [Online]. Available: <https://docs.micropython.org/>
- [13] QuestDB, *InfluxDB Line Protocol*, 2026, accessed: 2026-07-27. [Online]. Available: <https://questdb.com/docs/reference/api/ilp/overview/>
- [14] Litestar Framework, *Litestar Documentation*, 2026, accessed: 2026-07-27. [Online]. Available: <https://docs.litestar.dev/>
- [15] B. V. de Ven *et al.*, "Bokeh: Python library for interactive visualization," in *Proc. 14th Python in Science Conf.*, 2015, pp. 1–5. [Online]. Available: https://conference.scipy.org/proceedings/scipy2015/bryan_van_de_ven.html
- [16] Open Source Hardware Association, "Open source hardware definition 1.0," 2026, accessed: 2026-07-27. [Online]. Available: <https://www.oshwa.org/definition/>
- [17] Aosong Electronics Co., Ltd., *DHT20 Digital Temperature and Humidity Sensor Datasheet*, 2021, accessed: 2026-07-27. [Online]. Available: <https://www.adafruit.com/product/5183>
- [18] TAOS, Inc., *TSL2560 and TSL2561 Light-to-Digital Converter Datasheet*, 2009. [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/TSL2561.pdf>
- [19] Plantower, *PMS5003 Series Data Manual*, 2016. [Online]. Available: https://cdn-shop.adafruit.com/product-files/3686/plantower-pms5003-manual_v2.3.pdf
- [20] Zhengzhou Winsen Electronics Technology Co., Ltd., *MH-Z19B NDIR CO2 Module User's Manual*, 2019. [Online]. Available: https://www.winsen-sensor.com/d/files/infrared-gas-sensor/mh-z19b-co2-ver1_0.pdf
- [21] Docker, Inc., *Compose Specification*, 2026, accessed: 2026-07-27. [Online]. Available: <https://docs.docker.com/reference/compose-file/>